

[Flink] AI 辅助编写需求代码、解决代码 Bug

练习题

提示：注意版本

flink 是一个版本发布较快、特性变化幅度较大的实时计算框架，尤其是在状态和 SQL 两个模块上变动最为明显。因此，当你使用 GPT 来进行 flink 相关的提问时，一定要格外注意你所使用的 flink 版本。截至 2023 年 4 月 19 日，ChatGPT 的两个常用模型（GPT3.5 和 GPT4）所了解的最新 flink 版本是 1.14。另外，由于 flink 的特性变动较大，为了获得高质量的回答，最好在提问时明确指出所使用的版本号。

习题一 编写 SourceFunction 模拟数据

在企业开发环境中，有时数据仅存储在内部网络中，因此编写需求只能在公司电脑上完成，开发和测试也可能会遇到不小的问题。在这种情况下，你可以选择编写一个 SourceFunction 来模拟数据，只需确保数据格式正确即可。这样，你就可以在自己的电脑上编写后续的计算逻辑。现在，你可以使用 GPT 快速创建一个 SourceFunction 来生成模拟数据。

下面是一个可供参考的数据结构{

```
id: int,  
  
list: [{  
  
code: int,  
  
value: double  
  
}],  
  
...
```

```
],  
  
    ts: long  
  
}
```

id 字段是设备的 id, 生成 10 个设备

list 里是设备 4 个温度传感器, code 为 {0, 1, 2, 3}

value 是设备的温度, 在 30-60 度之间

ts 是数据上报的毫秒时间戳

习题二 开窗聚合

依赖于习题一

有了之前的 SourceFunction, 我们就可以在其基础之上写一些计算逻辑。现在可以尝试在 source 的后面补充一些计算逻辑, 比如, 求出每个设备的温度的最大值。

习题三 分析和处理异常

下面是一段 flink 程序报出的异常

```
2023-05-29 14:53:02  
  
java.lang.Exception: Exception while creating StreamOperatorStateContext.  
    at  
org.apache.flink.streaming.api.operators.StreamTaskStateInitializerImpl.streamOperatorStateContext(StreamTaskStateInitializerImpl.java:254)  
    at  
org.apache.flink.streaming.api.operators.AbstractStreamOperator.initializeState(AbstractStreamOperator.java:272)  
    at  
org.apache.flink.streaming.runtime.tasks.OperatorChain.initializeStateAndOpenOperators(OperatorChain.java:432)  
    at
```

```
org.apache.flink.streaming.runtime.tasks.StreamTask.lambda$beforeInvoke$2(StreamTask.java:
545)
    at
org.apache.flink.streaming.runtime.tasks.StreamTaskActionExecutor$1.runThrowing(StreamTask
ActionExecutor.java:50)
    at
org.apache.flink.streaming.runtime.tasks.StreamTask.beforeInvoke(StreamTask.java:535)
    at org.apache.flink.streaming.runtime.tasks.StreamTask.invoke(StreamTask.java:575)
    at org.apache.flink.runtime.taskmanager.Task.doRun(Task.java:758)
    at org.apache.flink.runtime.taskmanager.Task.run(Task.java:573)
    at java.lang.Thread.run(Thread.java:748)
Caused by: org.apache.flink.util.FlinkException: Could not restore operator state backend
for CoBroadcastWithNonKeyedOperator_d10e660091c173e8fb1a17070123b4ab_(2/2) from any of the
1 provided restore options.
    at
org.apache.flink.streaming.api.operators.BackendRestorerProcedure.createAndRestore(Backend
RestorerProcedure.java:160)
    at
org.apache.flink.streaming.api.operators.StreamTaskStateInitializerImpl.operatorStateBacke
nd(StreamTaskStateInitializerImpl.java:285)
    at
org.apache.flink.streaming.api.operators.StreamTaskStateInitializerImpl.streamOperatorStat
eContext(StreamTaskStateInitializerImpl.java:173)
    ... 9 more
Caused by: org.apache.flink.runtime.state.BackendBuildingException: Failed when trying to
restore operator state backend
    at
org.apache.flink.runtime.state.DefaultOperatorStateBackendBuilder.build(DefaultOperatorSta
teBackendBuilder.java:83)
    at
org.apache.flink.runtime.state.filesystem.FsStateBackend.createOperatorStateBackend(FsStat
eBackend.java:576)
    at
org.apache.flink.streaming.api.operators.StreamTaskStateInitializerImpl.lambda$operatorSta
teBackend$0(StreamTaskStateInitializerImpl.java:276)
    at
```

```
org.apache.flink.streaming.api.operators.BackendRestorerProcedure.attemptCreateAndRestore(BackendRestorerProcedure.java:168)
    at
org.apache.flink.streaming.api.operators.BackendRestorerProcedure.createAndRestore(BackendRestorerProcedure.java:135)
    ... 11 more
Caused by: java.lang.IllegalArgumentException: Can not set java.lang.String field
com.atguigu.dw.bean.odsToDwd.TableProcess.operateType to java.util.ArrayList
    at
sun.reflect.UnsafeFieldAccessorImpl.throwSetIllegalArgumentException(UnsafeFieldAccessorImpl.java:167)
    at
sun.reflect.UnsafeFieldAccessorImpl.throwSetIllegalArgumentException(UnsafeFieldAccessorImpl.java:171)
    at sun.reflect.UnsafeFieldAccessorImpl.ensureObj(UnsafeFieldAccessorImpl.java:58)
    at
sun.reflect.UnsafeObjectFieldAccessorImpl.set(UnsafeObjectFieldAccessorImpl.java:75)
    at java.lang.reflect.Field.set(Field.java:764)
    at
org.apache.flink.api.java.typeutils.runtime.PojoSerializer.initializeFields(PojoSerializer.java:205)
    at
org.apache.flink.api.java.typeutils.runtime.PojoSerializer.deserialize(PojoSerializer.java:388)
    at
org.apache.flink.runtime.state.OperatorStateRestoreOperation.deserializeBroadcastStateValues(OperatorStateRestoreOperation.java:244)
    at
org.apache.flink.runtime.state.OperatorStateRestoreOperation.restore(OperatorStateRestoreOperation.java:185)
    at
org.apache.flink.runtime.state.DefaultOperatorStateBackendBuilder.build(DefaultOperatorStateBackendBuilder.java:80)
    ... 15 more
```

请分析一下这个异常的原因

习题四 代码优化

下面是一个用 scala+flink 实现的开窗函数，这段代码上线后，运行 2~3 天，就会因为内存溢出挂掉，请分析一下这个开窗函数有什么问题。

```
class MyWindowFunction2 extends
ProcessWindowFunction[(StorageLog,String),StorageLog,String,TimeWindow]{

    var maxSizeMap : MapState[String,StorageLog]=_

    override def open(parameters: Configuration): Unit = {
        val vs = new MapStateDescriptor[String,StorageLog]("maxSizeMap",classOf[String],
classOf[StorageLog])
        maxSizeMap= getRuntimeContext.getMapState(vs)
    }

    override def process(ip:String, context: Context, elements:
Iterable[(StorageLog,String)], out: Collector[StorageLog]): Unit = {
        for (elem <- elements) {
            val source: String = elem._1.source
            val fileName: String = elem._1.fileName
            val mapKey=source+fileName
            val fileSize: Long = elem._1.fileSize
            if (!maxSizeMap.contains(mapKey)){
                maxSizeMap.put(mapKey,elem._1)
            }else{
                val mapSize: Long = maxSizeMap.get(mapKey).fileSize
                if(fileSize>mapSize){
                    maxSizeMap.put(mapKey,elem._1)
                }
            }
        }
        val mapIt: util.Iterator[Map.Entry[String, StorageLog]] = maxSizeMap.iterator()
        while (mapIt.hasNext) {
```

```
    val value: Map.Entry[String, StorageLog] = mapIt.next()
    //          nObject.put(value.getKey, value.getValue)
    out.collect(value.getValue)
  }

}

}
```

参考案

习题一 编写 SourceFunction 模拟数据

第 1 轮提示

提示

帮我用 java 代码写一个 flink 里的 sourceFunction 来生成数据。数据格式如下

```
{

  id: int,

  list: [{

    code: int,

    value: double

  },

  ...

],

  ts: long
```

```
}
```

id 字段是设备的 id, 生成 10 个设备

list 里是设备 4 个温度传感器, code 为 {0, 1, 2, 3}

value 是设备的温度, 在 30-60 度之间

ts 是数据上报的毫秒时间戳

答

根据您的需求, 这里是一个使用 Java 编写的 Flink SourceFunction, 它会生成符合您所描述的数据格式的数据:

首先, 定义一个 **SensorData** 类用于存储数据:

```
import java.util.List;

public class SensorData {
    public int id;
    public List<Sensor> list;
    public long ts;

    public SensorData(int id, List<Sensor> list, long ts) {
        this.id = id;
        this.list = list;
        this.ts = ts;
    }

    public static class Sensor {
        public int code;
        public double value;

        public Sensor(int code, double value) {
            this.code = code;
            this.value = value;
        }
    }
}
```

```
    }  
    }  
}
```

接下来，实现一个 Flink SourceFunction:

```
import org.apache.flink.streaming.api.functions.source.SourceFunction;  
import java.util.ArrayList;  
import java.util.Random;  
  
public class SensorDataSourceFunction implements SourceFunction<SensorData> {  
  
    private volatile boolean running = true;  
    private final int numDevices = 10;  
    private final int numSensors = 4;  
    private final Random random = new Random();  
  
    @Override  
    public void run(SourceContext<SensorData> ctx) throws Exception {  
        while (running) {  
            for (int deviceId = 0; deviceId < numDevices; deviceId++) {  
                ArrayList<SensorData.Sensor> sensorList = new ArrayList<>();  
                for (int sensorCode = 0; sensorCode < numSensors; sensorCode++) {  
                    double temperature = 30 + random.nextDouble() * 30;  
                    sensorList.add(new SensorData.Sensor(sensorCode, temperature));  
                }  
  
                long timestamp = System.currentTimeMillis();  
                SensorData data = new SensorData(deviceId, sensorList, timestamp);  
                ctx.collect(data);  
  
                Thread.sleep(1000);  
            }  
        }  
    }  
}
```



```

    }

    @Override
    public void cancel() {
        running = false;
    }
}

```

然后，你可以在你的 Flink 程序中使用这个 **SensorDataSourceFunction**:

```

import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;

public class SensorDataGenerator {

    public static void main(String[] args) throws Exception {
        final StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();

        DataStream<SensorData> sensorDataStream = env.addSource(new
SensorDataSourceFunction());

        // 在这里添加你的 Flink 算子，如 map、filter 等
        // 例如: sensorDataStream.map(...).filter(...).sink(...);

        env.execute("Sensor Data Generator");
    }
}

```

这个程序会创建一个 SourceFunction，每秒生成 10 个设备的数据，每个设备包含 4 个温度传感器，温度值在 30-60 度之间，还包含一个时间戳。当然，您可以根据需要修改这个程序。

反馈

这个生成器已经非常完善了，现在我们只需要为 Bean 类添加一个可读性良好的 toString 方法，并在 main 方法中的 sensorDataStream 对象上添加一个 print 算子，就可以将模拟数据打印到控制台了。

下面是代码运行后的效果：

```
E:\Program\jshell\jshell.exe ...
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
15> SensorData[id=0, list=[Sensor[code=0, value=32.469528477542305], Sensor[code=1, value=44.5828958218675], Sensor[code=2, value=46.7852658685877], Sensor[code=3, value=40.68223819946845]], ts=1682842245170)
16> SensorData[id=1, list=[Sensor[code=0, value=51.29512811613751], Sensor[code=1, value=46.588292895241624], Sensor[code=2, value=45.17649529419906], Sensor[code=3, value=45.36418641649745]], ts=1682842246327)
17> SensorData[id=2, list=[Sensor[code=0, value=33.25051466642576], Sensor[code=1, value=41.22814903559572], Sensor[code=2, value=33.18753292394924], Sensor[code=3, value=56.00152146778995]], ts=1682842247539)
18> SensorData[id=3, list=[Sensor[code=0, value=41.8841895141986], Sensor[code=1, value=54.5762489974683], Sensor[code=2, value=45.519391595444256], Sensor[code=3, value=39.41671592829495]], ts=1682842248353)
19> SensorData[id=4, list=[Sensor[code=0, value=34.49980679590152], Sensor[code=1, value=51.15491867982091], Sensor[code=2, value=39.58853579615537], Sensor[code=3, value=34.52727488815189]], ts=1682842249366)
20> SensorData[id=5, list=[Sensor[code=0, value=48.98026529720862], Sensor[code=1, value=38.94328899734289], Sensor[code=2, value=57.19641399900321], Sensor[code=3, value=57.744907721285214]], ts=1682842250179)
1> SensorData[id=6, list=[Sensor[code=0, value=58.71806219997772], Sensor[code=1, value=63.554371914467126], Sensor[code=2, value=54.8421070456898], Sensor[code=3, value=35.88449723694895]], ts=1682842251391)
2> SensorData[id=7, list=[Sensor[code=0, value=43.78575784478156], Sensor[code=1, value=54.509595968433614], Sensor[code=2, value=54.349657677048626], Sensor[code=3, value=56.54594078286788]], ts=1682842252393)
Process finished with exit code 138
```

现在，这个 SourceFunction 已经可以生成我们想要的数​​据，它会每秒生成一条数据，如果你觉得慢的话，可以选择将 Thread.sleep(1000) 这行代码移到外面的 for 循环中，这样一秒可以为 10 个设备各生成 1 条数据。

```
@Override
public void run(SourceContext<SensorData> ctx) throws Exception {
    while (running) {
        for (int deviceId = 0; deviceId < numDevices; deviceId++) {
            ArrayList<SensorData.Sensor> sensorList = new ArrayList<>();
            for (int sensorCode = 0; sensorCode < numSensors; sensorCode++) {
                double temperature = 30 + random.nextDouble() * 30;
                sensorList.add(new SensorData.Sensor(sensorCode, temperature));
            }

            long timestamp = System.currentTimeMillis();
            SensorData data = new SensorData(deviceId, sensorList, timestamp);
            ctx.collect(data);

            Thread.sleep(1000);
        }
    }
}
```

现在，我们可以给数据生成器再加一个需求

第 2 轮提示

提示

一旦有一个传感器的温度超过 65 度，那么就算是异常数据，现在我希望这个数据生成器会有 5%的概率出现异常数据，该如何修改

答

要在数据生成器中添加 5%的概率生成异常数据，您可以修改 `SensorDataSourceFunction` 的 `run` 方法。在生成温度值之前，我们可以使用随机数来决定是否生成异常数据。这里是修改后的 `SensorDataSourceFunction`:

```
import org.apache.flink.streaming.api.functions.source.SourceFunction;
import java.util.ArrayList;
import java.util.Random;

public class SensorDataSourceFunction implements SourceFunction<SensorData> {

    private volatile boolean running = true;
    private final int numDevices = 10;
    private final int numSensors = 4;
    private final Random random = new Random();

    @Override
    public void run(SourceContext<SensorData> ctx) throws Exception {
        while (running) {
            for (int deviceId = 0; deviceId < numDevices; deviceId++) {
                ArrayList<SensorData.Sensor> sensorList = new ArrayList<>();
                for (int sensorCode = 0; sensorCode < numSensors; sensorCode++) {
                    double temperature;
                    // 5%的概率生成异常数据
                    if (random.nextDouble() < 0.05) {
                        temperature = 65 + random.nextDouble() * 5;
                    } else {
```

```

        temperature = 30 + random.nextDouble() * 30;
    }
    sensorList.add(new SensorData.Sensor(sensorCode, temperature));
}

long timestamp = System.currentTimeMillis();
SensorData data = new SensorData(deviceId, sensorList, timestamp);
ctx.collect(data);

Thread.sleep(1000);
}
}

@Override
public void cancel() {
    running = false;
}
}

```

现在，每个传感器的温度值有 5% 的概率超过 65 度，生成异常数据。其他部分代码保持不变。

反馈

现在，我们有了一个能在一定概率下生成异常数据的数据生成器。

习题二 开窗聚合

第 1 轮提示

提示

我想在之前的 `sourceFunction` 上写一个 flink 的开窗聚合，统计每个设备最近 30 秒的最高温度

答

要实现这个功能，你需要在 `KeyedStream` 上定义一个时间窗口，并使用一个聚合函数来找出窗口中的最高温度。以下是一个简单的 Java 代码示例，该代码将使用之前的 `SensorSource` 类作为数据源，并找出每个设备最近 30 秒的最高温度：

```
import org.apache.flink.api.common.functions.AggregateFunction;
import org.apache.flink.api.common.functions.RichFlatMapFunction;
import org.apache.flink.api.java.tuple.Tuple3;
import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.flink.streaming.api.windowing.assigners.TumblingProcessingTimeWindows;
import org.apache.flink.streaming.api.windowing.time.Time;
import org.apache.flink.util.Collector;

public class DeviceMaxTemperature {

    public static void main(String[] args) throws Exception {
        StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();

        DataStream<DeviceData> source = env.addSource(new SensorSource());

        source.flatMap(new FlatMapToDeviceReading())
            .keyBy(0)
```

```

        .window(TumblingProcessingTimeWindows.of(Time.seconds(30)))
        .aggregate(new MaxTemperatureAggregator())
        .print();

    env.execute("Device Max Temperature");
}

public static class FlatMapToDeviceReading extends RichFlatMapFunction<DeviceData,
Tuple3<Integer, Double, Long>> {
    @Override
    public void flatMap(DeviceData deviceData, Collector<Tuple3<Integer, Double,
Long>> collector) {
        for (SensorData sensorData : deviceData.list) {
            collector.collect(Tuple3.of(deviceData.id, sensorData.value,
deviceData.ts));
        }
    }
}

public static class MaxTemperatureAggregator implements
AggregateFunction<Tuple3<Integer, Double, Long>, Double, Double> {
    @Override
    public Double createAccumulator() {
        return Double.MIN_VALUE;
    }

    @Override
    public Double add(Tuple3<Integer, Double, Long> value, Double accumulator) {
        return Math.max(value.f1, accumulator);
    }

    @Override
    public Double getResult(Double accumulator) {
        return accumulator;
    }
}

```

```
@Override  
public Double merge(Double a, Double b) {  
    return Math.max(a, b);  
}  
}  
}
```

在这段代码中：

- `main` 方法中定义了 Flink 的执行环境，并且使用了我们之前创建的 `SensorSource` 作为数据源。
- `flatMap` 操作将设备数据转换为三元组，包含设备 ID，温度值和时间戳。
- 使用 `keyBy` 方法根据设备 ID 进行分区。
- `window` 方法定义了一个 30 秒的滚动窗口。
- `aggregate` 方法使用了一个自定义的聚合函数 `MaxTemperatureAggregator` 来找出每个窗口内的最大温度。
- `print` 方法将结果输出到控制台。

`FlatMapToDeviceReading` 类是一个 `RichFlatMapFunction`，用于将设备数据转换为三元组。

`MaxTemperatureAggregator` 类是一个 `AggregateFunction`，用于找出温度的最大值。

反馈

可以看到 GPT 给出了合理的聚合代码。

习题三 分析和处理异常

第 1 轮提示

提示

2023-05-29 14:53:02

```
java.lang.Exception: Exception while creating StreamOperatorStateContext.
```

```
at
```

```
org.apache.flink.streaming.api.operators.StreamTaskStateInitializerImpl.streamOperatorStateContext(StreamTaskStateInitializerImpl.java:254)
```

```
at
```

```
org.apache.flink.streaming.api.operators.AbstractStreamOperator.initializeState(AbstractStreamOperator.java:272)
```

```
at
```

```
org.apache.flink.streaming.runtime.tasks.OperatorChain.initializeStateAndOpenOperators(OperatorChain.java:432)
```

```
at
```

```
org.apache.flink.streaming.runtime.tasks.StreamTask.lambda$beforeInvoke$2(StreamTask.java:545)
```

```
at
```

```
org.apache.flink.streaming.runtime.tasks.StreamTaskActionExecutor$1.runThrowing(StreamTaskActionExecutor.java:50)
```

```
at
```

```
org.apache.flink.streaming.runtime.tasks.StreamTask.beforeInvoke(StreamTask.java:535)
```

```
at org.apache.flink.streaming.runtime.tasks.StreamTask.invoke(StreamTask.java:575)
```


at org.apache.flink.runtime.taskmanager.Task.doRun(Task.java:758)

at org.apache.flink.runtime.taskmanager.Task.run(Task.java:573)

at java.lang.Thread.run(Thread.java:748)

Caused by: org.apache.flink.util.FlinkException: Could not restore operator state backend for CoBroadcastWithNonKeyedOperator_d10e660091c173e8fb1a17070123b4ab_(2/2) from any of the 1 provided restore options.

at

org.apache.flink.streaming.api.operators.BackendRestorerProcedure.createAndRestore(BackendRestorerProcedure.java:160)

at

org.apache.flink.streaming.api.operators.StreamTaskStateInitializerImpl.operatorStateBackend(StreamTaskStateInitializerImpl.java:285)

at

org.apache.flink.streaming.api.operators.StreamTaskStateInitializerImpl.streamOperatorStateContext(StreamTaskStateInitializerImpl.java:173)

... 9 more

Caused by: org.apache.flink.runtime.state.BackendBuildingException: Failed when trying to restore operator state backend

at

org.apache.flink.runtime.state.DefaultOperatorStateBackendBuilder.build(DefaultOperatorStateBackendBuilder.java:83)

at

org.apache.flink.runtime.state.filesystem.FsStateBackend.createOperatorStateBackend(FsStateBackend.java:576)

at

org.apache.flink.streaming.api.operators.StreamTaskStateInitializerImpl.lambda\$operatorStateBackend\$0(StreamTaskStateInitializerImpl.java:276)

at

org.apache.flink.streaming.api.operators.BackendRestorerProcedure.attemptCreateAndRestore(BackendRestorerProcedure.java:168)

at

org.apache.flink.streaming.api.operators.BackendRestorerProcedure.createAndRestore(BackendRestorerProcedure.java:135)

... 11 more

Caused by: java.lang.IllegalArgumentException: Can not set java.lang.String field com.atguigu.dw.bean.odsToDwd.TableProcess.operateType to java.util.ArrayList

at

sun.reflect.UnsafeFieldAccessorImpl.throwSetIllegalArgumentException(UnsafeFieldAccessorImpl.java:167)

at

sun.reflect.UnsafeFieldAccessorImpl.throwSetIllegalArgumentException(UnsafeFieldAccessorImpl.java:171)

at sun.reflect.UnsafeFieldAccessorImpl.ensureObj(UnsafeFieldAccessorImpl.java:58)

at

sun.reflect.UnsafeObjectFieldAccessorImpl.set(UnsafeObjectFieldAccessorImpl.java:75)

at java.lang.reflect.Field.set(Field.java:764)

at

org.apache.flink.api.java.typeutils.runtime.PojoSerializer.initializeFields(PojoSerializer.java:205)

```
at
org.apache.flink.api.java.typeutils.runtime.PojoSerializer.deserialize(PojoSeriali
zer.java:388)

at
org.apache.flink.runtime.state.OperatorStateRestoreOperation.deserializeBroadcastS
tateValues(OperatorStateRestoreOperation.java:244)

at
org.apache.flink.runtime.state.OperatorStateRestoreOperation.restore(OperatorState
RestoreOperation.java:185)

at
org.apache.flink.runtime.state.DefaultOperatorStateBackendBuilder.build(DefaultOpe
ratorStateBackendBuilder.java:80)

... 15 more
```

我写的 flink 程序报了上面的异常，为什么

答

根据你提供的堆栈跟踪信息，看起来 Flink 程序在尝试恢复状态时出现了问题。具体的错误在这行：`java.lang.IllegalArgumentException: Can not set java.lang.String field com.txxy.rt.dw.bean.odsToDwd.TableProcess.operateType to java.util.ArrayList。`

这个错误表明 Flink 试图将一个 `java.util.ArrayList` 对象赋给了一个期望 `java.lang.String` 类型的字段 `com.txxy.rt.dw.bean.odsToDwd.TableProcess.operateType`。这是不可能的，因为 `ArrayList` 和 `String` 是不兼容的类型。

原因可能是：

1. 保存状态的序列化和反序列化过程中存在问题。可能在保存状态的时候，`operateType` 字段被错误地序列化为 `ArrayList`。检查你的序列化和反序列化逻辑，确保它们是正确的。
2. 你可能改变了 `TableProcess` 类的定义，并且新的版本与保存状态的旧版本不兼容。如果 `TableProcess` 类的 `operateType` 字段曾经是一个 `ArrayList`，但后来被改为了 `String`，而状态是在这个字段是 `ArrayList` 的情况下被保存的，那么就会出现这个问题。

为了解决这个问题，你可以：

1. 检查 `TableProcess` 类的 `operateType` 字段的类型和你保存的状态是否一致。确保 `TableProcess` 类定义的版本和你试图恢复的状态版本一致。
2. 检查序列化和反序列化逻辑，确保它们能正确地处理 `TableProcess` 类。
3. 如果无法更改代码以使其与旧状态兼容，你可能需要清除旧的状态并重新开始。但是这样做会丢失所有保存的状态信息。

请根据具体的代码和情况进行检查。

反馈

这个分析也是完全正确的。

习题四 代码优化

第 1 轮提示

提示

```
class MyWindowFunction2 extends
ProcessWindowFunction[(StorageLog, String), StorageLog, String, TimeWindow] {

    var maxSizeMap : MapState[String, StorageLog]=_
```

```

override def open(parameters: Configuration): Unit = {

    val vs = new MapStateDescriptor[String, StorageLog] ("maxSizeMap", classOf[String],
classOf[StorageLog])

    maxSizeMap= getRuntimeContext.getMapState(vs)

}

override def process(ip:String, context: Context, elements:
Iterable[(StorageLog, String)], out: Collector[StorageLog]): Unit = {

    for (elem <- elements) {

        val source: String = elem._1.source

        val fileName: String = elem._1.fileName

        val mapKey=source+fileName

        val fileSize: Long = elem._1.fileSize

        if (!maxSizeMap.contains(mapKey)) {

            maxSizeMap.put (mapKey, elem._1)

        }else{

            val mapSize: Long = maxSizeMap.get(mapKey).fileSize

            if(fileSize>mapSize) {

                maxSizeMap.put (mapKey, elem._1)

            }

        }

    }

    val mapIt: util.Iterator[Map.Entry[String, StorageLog]] = maxSizeMap.iterator()

```

```
while (mapIt.hasNext) {  
  
    val value: Map.Entry[String, StorageLog] = mapIt.next()  
  
    // nObject.put(value.getKey, value.getValue)  
  
    out.collect(value.getValue)  
  
}  
  
}  
  
}
```

上面这段代码有什么问题吗

答

你的 `MyWindowFunction2` 中的代码看起来在实现以下功能：在一个窗口内，根据每个 `StorageLog` 的 `source` 和 `fileName` 字段的组合，找出 `fileSize` 最大的 `StorageLog`。

这个实现基本上是正确的，但这里有一些问题和改进点：

1. **状态存储的清理：**你在状态存储中放入了数据，但从你的代码来看，你并没有在窗口处理完成后清理这个状态存储。这意味着随着时间的推移，状态存储的大小可能会无限增长，最终导致内存耗尽。你应该在处理完一个窗口后清理状态存储。
2. **MapState 的 Key 使用：**你在 `MapState` 中的 key 是由 `source` 和 `fileName` 组成，这意味着如果有两个 `StorageLog` 的 `source` 和 `fileName` 相同，但其他字段不同，它们会被视为同一个对象。这可能并不是你想要的。

针对这两点，你可以对代码进行如下调整：

```
override def process(ip:String, context: Context, elements: Iterable[(StorageLog,String)],  
out: Collector[StorageLog]): Unit = {  
    for (elem <- elements) {
```

```

    val source: String = elem._1.source
    val fileName: String = elem._1.fileName
    val mapKey=source+fileName
    val fileSize: Long = elem._1.fileSize
    if (!maxSizeMap.contains(mapKey)) {
        maxSizeMap.put(mapKey, elem._1)
    } else {
        val mapSize: Long = maxSizeMap.get(mapKey).fileSize
        if (fileSize > mapSize) {
            maxSizeMap.put(mapKey, elem._1)
        }
    }
}

val mapIt: util.Iterator[Map.Entry[String, StorageLog]] = maxSizeMap.iterator()
while (mapIt.hasNext) {
    val value: Map.Entry[String, StorageLog] = mapIt.next()
    out.collect(value.getValue)
    // Clear the state after using it
    maxSizeMap.remove(value.getKey)
}
}

```

这样，在每次处理窗口数据之后，都会清理掉状态存储的数据，防止无限制地消耗内存。

此外，你也需要检查 **source**、**fileName** 和 **fileSize** 是否可能为 **null**，因为如果它们是 **null**，可能会引发空指针异常

反馈

GPT 一下就发现了这段代码存在着潜在的 OOM 风险，并重写了 **process** 方法。现在，它会在窗口聚合的计算逻辑完成后，去清理状态里用不到的键值对。